

파이썬을 이용한 웹 데이터 수집

웹 데이터 수집 방법

- 데이터 제공 사이트에서 파일로 다운로드
- 크롤링(스크래핑)을 통한 수집
- 오픈API를 이용하여 수집

1. 웹 크롤링(Web Crawling)

웹 크롤링 개념

- 웹의 정보를 자동으로 수집하는 것
- 웹 페이지의 링크 구조를 탐색하고, 웹사이트의 다양한 페이지를 방문하며 데이터를 포괄적으로 수집 및 저장하는 작업
- 웹 크롤러(crawler) : 웹의 정보를 수집하기 위해 만든 프로그램
- 웹 페이지의 링크를 통해 계속해서 정보를 찾아 나감
 - 중복된 데이터 수집이 될 수 있으므로 중복된 데이터 처리가 필수
- 웹 스크래핑(scraping) : 특정 웹 페이지에서 필요한 정보를 추출하는 작업

웹 크롤링의 활용 예

- 기업의 경우 웹 크롤링 과정을 통해 고객과 제품의 정보를 다양한 소셜미디어 플랫폼에서 웹 크롤링을 통해 정보를 수집하고 브랜드 성과와 고객 만족도를 파악
- 주식시장 : 웹 크롤링을 통해 다양한 플랫폼에서 원하는 기간의 주가 데이터 수집하여 트렌드 발견 및 미래 주가 예측 모델에 활용
- 부동산 시장 : 부동산 가격 예측과 책정에 활용
 - 미국의 부동산 거래 플랫폼 질로우(zillow) : 웹 크롤링을 통해 정보를 수집하여 부동산 가격을 평가
 - <https://zillow.com>

웹 크롤링 허용 문제

- 모든 사이트는 웹 크롤링 권한에 관해 명시한 페이지가 존재함
- robots.txt 파일 : 크롤링을 위한 규칙 명시한 페이지
- 사이트 URL 끝에 robots.txt 를 붙이면 크롤링 권한에 관한 내용 확인

```
User-agent: *  
Disallow: /owner  
Disallow: /guestbook  
Disallow: /m/guestbook  
Disallow: /manage  
Disallow: /admin  
Disallow: /oldadmin  
Disallow: /search  
Disallow: /m/search  
Disallow: /m/admin  
Disallow: /like  
Allow: /  
  
User-agent: Mediapartners-Google  
Allow: /  
  
User-agent: bingbot  
Crawl-delay: 30
```

- 'Disallow' : 크롤링 허용하지 않는 경로
- 'Allow' : 크롤링 허용 경로

예. <http://www.google.com/robots.txt>

- 크롤링 전에 항상 robots.txt 파일을 확인할 것

2. 파이썬을 이용한 웹 크롤링

2.1 웹크롤링 준비

크롤링하는 웹사이트 살펴보기

- <http://www.naver.com>

기본 모듈 임포트

```
In [1]: import requests
```

requests 모듈

- Python에서 HTTP 요청을 보내는 모듈
- URL 주소에 있는 내용을 요청할 때 사용
 - HTTP request types: GET, PUT, DELETE, HEAD and OPTIONS
- <https://requests.readthedocs.io/en/latest/>
- <https://requests.readthedocs.io/en/latest/user/quickstart/>
- <https://requests.readthedocs.io/en/latest/api/>

requests 모듈의 get() 함수

형식 : request(method, url, **kwargs)

- 지정된 url을 요청하면 response 객체 반환

```
In [2]: url = 'https://www.naver.com'
response = requests.get(url)
response
```

Out[2]: <Response [200]>

response 객체의 필드들

status-code

- Integer Code of responded HTTP Status
- 예. 404 or 200

```
In [3]: type(response)
```

Out[3]: requests.models.Response

```
In [4]: response.status_code
```

Out[4]: 200

headers

```
In [5]: response.headers
```

Out[5]: {'Date': 'Wed, 19 Aug 2026 05:11:48 GMT', 'content-type': 'text/html; charset=UTF-8', 'transfer-encoding': 'chunked', 'set-cookie': 'PM_CK_loc=1c11432d34ac25587130d8f9f84ea8d77ede3f75003a10551a3d602f71fba3dc; Expires=Thu, 20 Aug 2026 05:11:48 GMT; Path=/; HttpOnly', 'cache-control': 'no-cache, no-store, must-revalidate', 'pragma': 'no-cache', 'x-frame-options': 'DENY', 'content-encoding': 'gzip', 'strict-transport-security': 'max-age=63072000; includeSubdomains', 'referrer-policy': 'unsafe-url', 'server': 'nfront'}

```
In [6]: type(response.headers)
```

Out[6]: requests.structures.CaseInsensitiveDict

```
In [8]: list(response.headers.keys())
```

Out[8]: ['Date',
'content-type',
'transfer-encoding',
'set-cookie',
'cache-control',
'pragma',
'x-frame-options',
'content-encoding',
'strict-transport-security',
'referrer-policy',
'server']

```
In [9]: response.headers['Date']
```

Out[9]: 'Wed, 19 Aug 2026 05:11:48 GMT'

```
In [10]: response.headers['content-type']
```

Out[10]: 'text/html; charset=UTF-8'

encoding

```
In [11]: response.encoding
```

```
Out[11]: 'UTF-8'
```

text

- 브라우저가 사이트에 요청해서 받은 결과와 같은 결과

```
In [14]: # print(response.text)
```

```
In [16]: # response.content    # byte 문자열로 표현
```

예1.

request모듈의 get()을 이용하여 <http://www.tistory.com> 사이트 메인페이지 요청

```
In [17]: url = 'http://www.tistory.com'
response = requests.get(url)
print(f'상태코드: {response.status_code}')
```

상태코드: 200

```
In [19]: def request_url(url):
response = requests.get(url)
print(f'{url} 요청결과 상태코드: {response.status_code}')
```

예2.

request모듈의 get()을 이용하여 <https://www.google.co.kr> 사이트 메인페이지 요청

```
In [20]: url = 'https://www.google.co.kr'
request_url(url)
```

<https://www.google.co.kr> 요청결과 상태코드: 200

파라미터 전달 : requests.get(URL, params= {})

- 파라미터란? : 사이트의 문서를 요청할 때 서버로 전달되는 정보
- 함수의 파라미터 처럼 문서를 찾기위한 정보나 명령을 수행하기 위한 정보를 같이 전달하게 되는 데 그 정보를 파라미터라고 함
- <https://sports.news.naver.com/news?oid=144&aid=0000948747>
- params = {'param1': 'value1', 'param2': 'value'}
- res = requests.get(URL, params=params)

```
In [21]: url = 'https://sports.news.naver.com/news'
params = {'oid': '144', 'aid': '0000948747'}
response = requests.get(url, params=params)
response
```

```
Out[21]: <Response [200]>
```

```
In [23]: response.text[:500]
```



```
Out[23]: '<!DOCTYPE html><html lang="ko"><head><meta charset="utf-8"/><meta name="viewport" content="width=device-width, initial-scale=1, minimum-scale=1, maximum-scale=1, user-scalable=no"/><link rel="preload" href="https://ssl.pstatic.net/static.sports/nubes-resource/sports-web-next/news/260806-112452/_next/static/media/d587d1c112526568-s.p.woff2" as="font" crossorigin="" type="font/woff2"/><link rel="preload" href="https://ssl.pstatic.net/static.sports/nubes-resource/sports-web-next/news/260806-112452/'
```

```
In [25]: url = 'https://sports.news.naver.com/news'
params = {'oid': '144', 'aid': '0000948747'}
urls = f'{url}?oid={params["oid"]}&&aid={params["aid"]}'
urls
```

```
Out[25]: 'https://sports.news.naver.com/news?oid=144&&aid=0000948747'
```

2.2 웹크롤링

데이터 수집을 위한 절차

1. 문서 내용 읽어오기
 - **urllib.request 패키지** 사용 :
 - url을 넘겨주면 데이터를 텍스트 형태로 반환(기본 내장 패키지)
 - urlopen() : url 접속
 - read() : 데이터 읽어오기
 - text 속성을 사용해서 데이터 읽어올 수도 있음
2. 문서에서 원하는 내용 추출하기(파싱)
 - **BeautifulSoup 패키지** 사용
 - find() / findAll() 함수 사용

① 웹페이지 데이터 가져오기

urllib 모듈

- 웹에서 얻은 데이터를 다루는 파이썬 패키지
- 총 4개의 모듈로 구성
 - urllib.request : opening and reading URLs
 - urllib.error : containing the exceptions raised by urllib.request
 - urllib.parse : parsing URLs
 - urllib.robotparser : parsing robots.txt files
- <https://docs.python.org/3/library/urllib.html>

urllib.request

- 웹문서를 열어 데이터를 읽어오는 모듈
- higher-level HTTP client interface
- <https://docs.python.org/3/library/urllib.request.html>

웹 데이터를 다루기 위한 모듈 импорт

```
In [26]: from urllib.request import urlopen
```

urllib.request.urlopen() 함수

- Open the URL url, which can be either a string or a Request object

```
urllib.request.urlopen(url, data=None, [timeout, ], cafile=None, capath=None,
cadefault=False, context=None)
```

```
In [27]: url = 'https://www.naver.com'
response = urlopen(url)
response
```

```
Out[27]: <http.client.HTTPResponse at 0x1859836b490>
```

- HTTPResponse 객체에서 소스 코드 읽기 : read()

```
In [29]: # response.read() # 파이썬의 b'' 형식의 문자열 반환 -> 바이트(byte) 문자열(0~255 사이의 정수 ,
```

② 원하는 내용 추출하기(parsing)

- html 문서에서 필요한 내용만 추출
- **BeautifulSoup** 라이브러리 사용
- find() / find_all() 등 함수 사용

BeautifulSoup

- 데이터를 추출하는데 필요한 기능이 들어 있는 라이브러리 (파싱 라이브러리)
- 외부 라이브러리
- <https://beautiful-soup-4.readthedocs.io/en/latest/>
- <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

BeautifulSoup 패키지 설치

```
In [30]: import bs4
```

```
In [31]: bs4.__version__
```

```
Out[31]: '4.15.0'
```

```
In [ ]: # 설치되어 있지 않은 경우
# !pip install beautifulsoup4
```

bs4 파싱 객체 생성하기

- BeautifulSoup 패키지 импорт

```
In [32]: from bs4 import BeautifulSoup
```

- BeautifulSoup(접속객체, 파서기)

```
In [33]: url = 'https://www.naver.com'
html = urlopen(url)
soup = BeautifulSoup(html, 'html.parser')
```

```
In [34]: type(soup)
```

Out[34]: bs4.BeautifulSoup

- prettify()
 - 들여쓰기, 계층적인 구조로 출력
 - 가독성 높음

In [37]: `print(soup.prettify()[ :1000])`

```
<!DOCTYPE html>
<html class="fzoom" lang="ko">
  <head>
    <meta charset="utf-8"/>
    <meta content="origin" name="Referrer"/>
    <meta content="IE=edge" http-equiv="X-UA-Compatible"/>
    <meta content="width=1190" name="viewport"/>
    <title>
      NAVER
    </title>
    <meta content="NAVER" name="apple-mobile-web-app-title"/>
    <meta content="index,nofollow" name="robots"/>
    <meta content="네이버 메인에서 다양한 정보와 유용한 컨텐츠를 만나 보세요" name="description"/>
  >
    <meta content="네이버" property="og:title"/>
    <meta content="https://www.naver.com/" property="og:url"/>
    <meta content="https://s.pstatic.net/static/www/mobile/edit/2016/0705/mobile_212852414260.png" property="og:image"/>
    <meta content="네이버 메인에서 다양한 정보와 유용한 컨텐츠를 만나 보세요" property="og:description"/>
    <meta content="summary" name="twitter:card"/>
    <meta content="" name="twitter:title"/>
    <meta content="https://www.naver.com/" name="twitter:url"/>
    <meta content="https://s.pstatic.net/static/www/mobile/edit/2016/0705/mobile_212852414260.png" name="twi
```

특정 태그에서 텍스트 추출하기

: BeautifulSoup의 find(), find_all() 메서드 사용

- find(name, attrs, recursive, string, **kwargs)
- find_all(name, attrs, recursive, string, limit, **kwargs)

find() 메서드

- find(name, attrs, recursive, string, **kwargs)
 - attrs : {"속성": "값"}
- 찾은 요소의 첫번째 요소(element) 반환

In [38]: `html_str = '''<html>
 <head></head>
 <body>
 <div>Hello</div>
 <div>hi!</div>
 </body>
</html>
...'''`

In [39]: `soup = BeautifulSoup(html_str, 'html.parser')
soup`

```
Out[39]: <html>
<head></head>
<body>
<div>Hello</div>
<div>hi!</div>
</body>
</html>
```

```
In [40]: result = soup.find('div')
print(result, type(result))
```

```
<div>Hello</div> <class 'bs4.element.Tag'>
```

- find().text : 찾은 태그의 텍스트 가져오기

```
In [41]: result.text
```

```
Out[41]: 'Hello'
```

```
In [42]: soup.find('div').text
```

```
Out[42]: 'Hello'
```

find_all() 메서드

- find_all(name, attrs, recursive, string, **kwargs)
 - attrs : {"속성": "값"}
- 기준에 맞는 요소(element)를 모두 가져옴
- 리스트 타입을 반환

```
In [43]: html_str = """
<html>
  <body>
    <!-- 예. 찾은 태그들의 모든 text 가져오기 -->
    <ul>
      <li>hello</li>
      <li>bye</li>
      <li>welcome</li>
    </ul>
  </body>
</html>
"""
```

```
In [44]: soup = BeautifulSoup(html_str, 'html.parser')
```

```
In [45]: soup.find('ul')
```

```
Out[45]: <ul>
<li>hello</li>
<li>bye</li>
<li>welcome</li>
</ul>
```

```
In [46]: soup.find('li')
```

```
Out[46]: <li>hello</li>
```

```
In [47]: result = soup.find_all('li')
result
```

```
Out[47]: [<li>hello</li>, <li>bye</li>, <li>welcome</li>]
```

```
In [48]: result[0], type(result[1]), result[-1].text
```

```
Out[48]: (<li>hello</li>, bs4.element.Tag, 'welcome')
```

```
In [50]: for item in result:
         print(item.text)
```

```
hello
bye
welcome
```

```
In [51]: [item.text for item in result]
```

```
Out[51]: ['hello', 'bye', 'welcome']
```

```
In [52]: type(result)
```

```
Out[52]: bs4.element.ResultSet
```

해당 태그의 class 속성을 기준으로 찾기

```
In [49]: html_str = """
<html>
  <body>
    <ul class="greet">
      <li>hello</li>
      <li>bye</li>
      <li>welcome</li>
    </ul>
    <ul class="reply">
      <li>ok</li>
      <li>no</li>
      <li>sure</li>
    </ul>
  </body>
</html>
"""
```

```
In [53]: soup = BeautifulSoup(html_str, 'html.parser')
```

```
In [54]: soup.find_all('li')
```

```
Out[54]: [<li>hello</li>,
<li>bye</li>,
<li>welcome</li>,
<li>ok</li>,
<li>no</li>,
<li>sure</li>]
```

```
In [55]: [item.text for item in soup.find_all('li')]
```

```
Out[55]: ['hello', 'bye', 'welcome', 'ok', 'no', 'sure']
```

```
In [59]: [item.text for item in soup.find_all('ul')[0].find_all('li')]
```

```
Out[59]: ['hello', 'bye', 'welcome']
```

```
In [60]: soup.find('ul', {'class': 'greet'})
```

```
Out[60]: <ul class="greet">
         <li>hello</li>
         <li>bye</li>
         <li>welcome</li>
         </ul>
```

```
In [61]: soup.find('ul', {'class':'reply'})
```

```
Out[61]: <ul class="reply">
         <li>ok</li>
         <li>no</li>
         <li>sure</li>
         </ul>
```

```
In [63]: result = soup.find('ul', {'class':'reply'})
         for item in result.find_all('li'):
             print(item.text)
```

```
ok
no
sure
```

```
In [64]: result = soup.find('ul', {'class':'greet'})
         [item.text for item in result.find_all('li')]
```

```
Out[64]: ['hello', 'bye', 'welcome']
```

해당 태그의 id 속성을 기준으로 찾기

```
In [65]: html_str = """
         <html>
             <body>
                 <h1 id='title'>Hello Python</h1>
                 <p id="crawling">웹 크롤링</p>
                 <p id="parsing">파싱</p>
             </body>
         </html>
         """
```

```
In [66]: soup = BeautifulSoup(html_str, 'html.parser')
```

```
In [68]: soup.find('p', {'id':'parsing'}).text
```

```
Out[68]: '파싱'
```

```
In [70]: soup.find(id='parsing').text
```

```
Out[70]: '파싱'
```

```
In [71]: html_str = """
         <html>
             <head><title>크롤링 예제</title></head>
             <body>
                 <h1>파이썬 프로그래밍</h1>
                 <p>웹 페이지 분석</p>
                 <p>크롤링</p>
                 <p>파싱</p>
             </body>
         </html>
         """
```

```
In [73]: # p태그들의 내용 추출
soup = BeautifulSoup(html_str, 'html.parser')
result = [item.text for item in soup.find_all('p')]
result
```

```
Out[73]: ['웹 페이지 분석', '크롤링', '파싱']
```

```
In [74]: # 함수정의 : html, 태그이름, 속성이 주어지면 지정한 정보에 대한 컨텐츠 추출
def find_text_in_tags(html, tag_name, attrs=None):
    soup = BeautifulSoup(html, 'html.parser')
    if attrs:
        result = [tag.text for tag in soup.find_all(tag_name, attrs)]
    else:
        result = [tag.text for tag in soup.find_all(tag_name)]
    return result
```

```
In [75]: find_text_in_tags(html_str, 'p')
```

```
Out[75]: ['웹 페이지 분석', '크롤링', '파싱']
```

```
In [78]: html_str2 = """
<html>
  <body>
    <h1 id='title'>Hello Python</h1>
    <p id="crawling">웹 크롤링</p>
    <p id="parsing">파싱</p>
  </body>
</html>
"""
```

```
In [79]: find_text_in_tags(html_str2, 'p', {'id':'parsing'})
```

```
Out[79]: ['파싱']
```

태그 이름 사용하여 찾기

:.태그명

- .title
- .head
- .body
- .p

title 태그 찾기

```
In [80]: soup
```

```
Out[80]: <html>
<head><title>크롤링예제</title></head>
<body>
<h1>파이썬 프로그래밍</h1>
<p>웹 페이지 분석</p>
<p>크롤링</p>
<p>파싱</p>
</body>
</html>
```

```
In [81]: soup.title
```

```
Out[81]: <title>크롤링예제</title>
```

```
In [82]: soup.title.text
```

```
Out[82]: '크롤링예제'
```

head 태그 찾기

```
In [83]: soup.head, soup.head.text
```

```
Out[83]: (<head><title>크롤링예제</title></head>, '크롤링예제')
```

body 태그 찾기

```
In [84]: soup.body
```

```
Out[84]: <body>
<h1>파이썬 프로그래밍</h1>
<p>웹 페이지 분석</p>
<p>크롤링</p>
<p>파싱</p>
</body>
```

```
In [85]: soup.h1
```

```
Out[85]: <h1>파이썬 프로그래밍</h1>
```

```
In [86]: soup.p
```

```
Out[86]: <p>웹 페이지 분석</p>
```

형제, 부모, 자식(자손) 태그 찾기

- 속성 :
 - .children, .contents
 - .next_sibling, .next_siblings
 - .previous_sibling, .previous_siblings
 - .parent, .parents
- 메서드 :
 - find_next_siblings(name, attrs, string, limit, **kwargs)
 - find_next_sibling(name, attrs, string, **kwargs)
 - find_previous_sibling(name, attrs, string, **kwargs)
 - find_previous_siblings(name, attrs, string, limit, **kwargs)
 - find_parent(name, attrs, string, **kwargs)
 - find_parents(name, attrs, string, limit, **kwargs)

태그의 자손 태그 찾기 : .contents, .children

```
In [87]: soup
```



```
Out[87]: <html>
<head><title>크롤링 예제</title></head>
<body>
<h1>파이썬 프로그래밍</h1>
<p>웹 페이지 분석</p>
<p>크롤링</p>
<p>파싱</p>
</body>
</html>
```

```
In [88]: soup.body
```

```
Out[88]: <body>
<h1>파이썬 프로그래밍</h1>
<p>웹 페이지 분석</p>
<p>크롤링</p>
<p>파싱</p>
</body>
```

```
In [89]: soup.body.p
```

```
Out[89]: <p>웹 페이지 분석</p>
```

```
In [90]: soup.body.contents
```

```
Out[90]: ['\n',
<h1>파이썬 프로그래밍</h1>,
'\n',
<p>웹 페이지 분석</p>,
'\n',
<p>크롤링</p>,
'\n',
<p>파싱</p>,
'\n']
```

```
In [92]: list(soup.body.children)
```

```
Out[92]: ['\n',
<h1>파이썬 프로그래밍</h1>,
'\n',
<p>웹 페이지 분석</p>,
'\n',
<p>크롤링</p>,
'\n',
<p>파싱</p>,
'\n']
```

```
In [93]: for child in soup.body.children:
print(child)
```

```
<h1>파이썬 프로그래밍</h1>
```

```
<p>웹 페이지 분석</p>
```

```
<p>크롤링</p>
```

```
<p>파싱</p>
```

```
In [94]: for child in soup.body.children:
if child != '\n':
```

```
print(child)
```

```
<h1>파이썬 프로그래밍</h1>
<p>웹 페이지 분석</p>
<p>크롤링</p>
<p>파싱</p>
```

```
In [95]: [child.text for child in soup.body.children if child != '\n']
```

```
Out[95]: ['파이썬 프로그래밍', '웹 페이지 분석', '크롤링', '파싱']
```

형제 태그 찾기 : `.next_sibling`, `.next_siblings`

```
In [96]: soup
```

```
Out[96]: <html>
<head><title>크롤링예제</title></head>
<body>
<h1>파이썬 프로그래밍</h1>
<p>웹 페이지 분석</p>
<p>크롤링</p>
<p>파싱</p>
</body>
</html>
```

```
In [97]: list(soup.body.children)
```

```
Out[97]: ['\n',
<h1>파이썬 프로그래밍</h1>,
'\n',
<p>웹 페이지 분석</p>,
'\n',
<p>크롤링</p>,
'\n',
<p>파싱</p>,
'\n']
```

```
In [99]: soup.find('p').next_sibling
```

```
Out[99]: '\n'
```

```
In [100... soup.find('p').next_sibling.next_sibling
```

```
Out[100... <p>크롤링</p>
```

```
In [102... list(soup.find('p').next_siblings)
```

```
Out[102... ['\n', <p>크롤링</p>, '\n', <p>파싱</p>, '\n']
```

형제 태그 찾기2 : `.previous_sibling`, `.previous_siblings`

```
In [103... soup.find('p').previous_sibling
```

```
Out[103... '\n'
```

```
In [104... soup.find('p').previous_sibling.previous_sibling
```

```
Out[104... <h1>파이썬 프로그래밍</h1>
```

부모 태그 찾기 : `.parent`, `.parents`

```
In [105... soup.find('p').parent
```

```
Out[105... <body>
<h1>파이썬 프로그래밍</h1>
<p>웹 페이지 분석</p>
<p>크롤링</p>
<p>파싱</p>
</body>
```

```
In [107... list(soup.find('p').parents)
```

```
Out[107... [
```

```
In [108... soup.find('p').parent.name
```

```
Out[108... 'body'
```

```
In [109... soup.find('h1').parent.name
```

```
Out[109... 'body'
```

```
In [110... soup.find('body').parent.name
```

```
Out[110... 'html'
```

```
In [111... soup.find('html').parent.name
```

```
Out[111... '[document]'
```

```
In [112... for parent in soup.find('h1').parents:
        print(parent.name)
```

```
body
html
[document]
```

다음/이전 태그 찾기

- find_all_next(name, attrs, string, limit, **kwargs)
- find_next(name, attrs, string, **kwargs)
- find_all_previous(name, attrs, string, limit, **kwargs)

- find_previous(name, attrs, string, **kwargs)

```
In [118... html_str2 = """
<html>
  <body>
    <h1 id='title'>Hello Python</h1>
    <p id="crawling">웹 크롤링</p>
    <p id="parsing">파싱</p>
    <hr>
  </body>
</html>
"""

soup = BeautifulSoup(html_str2, 'html.parser')
```

```
In [119... start = soup.find('h1')
start.find_all_next('p')
```

```
Out[119... [<p id="crawling">웹 크롤링</p>, <p id="parsing">파싱</p>]
```

```
In [120... start.find_all_next()
```

```
Out[120... [<p id="crawling">웹 크롤링</p>, <p id="parsing">파싱</p>, <hr/>]
```

```
In [121... start.find_next('p')
```

```
Out[121... <p id="crawling">웹 크롤링</p>
```

```
In [122... start.find_all_previous()
```

```
Out[122... [<body>
  <h1 id="title">Hello Python</h1>
  <p id="crawling">웹 크롤링</p>
  <p id="parsing">파싱</p>
  <hr/>
</body>,
<html>
<body>
  <h1 id="title">Hello Python</h1>
  <p id="crawling">웹 크롤링</p>
  <p id="parsing">파싱</p>
  <hr/>
</body>
</html>]
```

```
In [123... start.find_previous()
```

```
Out[123... <body>
<h1 id="title">Hello Python</h1>
<p id="crawling">웹 크롤링</p>
<p id="parsing">파싱</p>
<hr/>
</body>
```

```
In [125... soup.find('p').find_previous()
```

```
Out[125... <h1 id="title">Hello Python</h1>
```

```
In [126... soup.find('p').find_all_previous()
```

```
Out[126...] [<h1 id="title">Hello Python</h1>,
<body>
<h1 id="title">Hello Python</h1>
<p id="crawling">웹 크롤링</p>
<p id="parsing">파싱</p>
<hr/>
</body>,
<html>
<body>
<h1 id="title">Hello Python</h1>
<p id="crawling">웹 크롤링</p>
<p id="parsing">파싱</p>
<hr/>
</body>
</html>]
```

태그의 속성값 추출하기

- a 태그의 href 속성

```
In [127...] html_str = """
<html>
  <body>
    <ul class="ko">
      <li><a href="https://www.naver.com/">네이버</a></li>
      <li><a href="https://www.daum.net/">다음</a></li>
    </ul>
    <ul class="sns">
      <li><a href="https://www.goole.com/">구글</a></li>
      <li><a href="https://www.facebook.net/">페이스북</a></li>
    </ul>
  </body>
</html>
"""
```

```
In [128...] soup = BeautifulSoup(html_str, 'html.parser')
```

```
In [131...] soup.find('a').text
```

```
Out[131...] '네이버 '
```

```
In [132...] a1 = soup.find('a')
a1
```

```
Out[132...] <a href="https://www.naver.com/">네이버</a>
```

```
In [133...] a1.text
```

```
Out[133...] '네이버 '
```

```
In [134...] type(a1)
```

```
Out[134...] bs4.element.Tag
```

```
In [135...] a1['href']
```

```
Out[135...] 'https://www.naver.com/'
```

```
In [138...] links = [a['href'] for a in soup.find_all('a')]
links
```

Out[138...

```
['https://www.naver.com/',  
'https://www.daum.net/',  
'https://www.goole.com/',  
'https://www.facebook.net/']
```

select() 메서드를 사용하여 찾기

- CSS 문법을 사용해 HTML 요소(element)를 선택하는 메서드
- find() / find_all() 보다 복잡한 HTML 구조를 간결하게 표현할 수 있는 장점
- 조건에 맞는 모든 요소를 리스트로 반환
- CSS Selector를 사용 |선택방법|CSS Selector|예| |---|---|---| |태그| 태그명 |div| |class| .클래스
명|.product| |id| #아이디|#content| |태그 + class| 태그.클래스 |div.product| |하위요소(자손)| A
B |ul a
.product .name| |직계자식| A>B |ul>li| |여러 요소| A,B |h1, h2| |속성| [속성] |[href]| |속성값| [속성
=값] |[type='text']| |n번째 요소| :nth-child() |:nth-child(2)|

```
In [2]: import requests  
from urllib.request import urlopen  
from bs4 import BeautifulSoup
```

```
In [1]: html_str = """  
<html>  
  <body>  
    <div id="wrap">  
      <div id="mainMenuBox">  
        <ul>  
          <li><a href="#">패션 잡화</a></li>  
          <li><a href="#">주방 용품</a></li>  
          <li><a href="#">생활 건강</a></li>  
          <li><a href="#">DIY가구</a></li>  
        </ul>  
      </div>  
      <div>  
        <table>  
          <tr>  
            <td></td>  
            <td></td>  
            <td></td></tr>  
          <tr id="prdName">  
            <td>솔로리스트<br>걸리쉬 리본단화</td>  
            <td>맥컬린<br>그레이가보시스트랩 펌프스</td>  
            <td>맥컬린<br>섹슈얼인사이드펌프스</td></tr>  
          <tr id="price">  
            <td>100,000원</td><td>200,000원</td><td>120,000원</td></tr>  
        </table>  
      </div>  
      <div id="out_box">  
        <div class="box">  
          <h4>공지사항</h4>  
          <hr>  
          <a href="#">[배송] : 무료배송 변경 안내 18.10.20</a><br>  
          <a href="#">[전시] : DIY 가구 전시 안내 18.10.31</a><br>  
          <a href="#">[판매] : 11월 특가 상품 안내 18.11.05</a>  
        </div>  
        <div class="box">  
          <h4>커뮤니티</h4>  
          <hr>  
          <a href="#">[레시피] : 살 안찌는 야식 만들기</a><br>  
          <a href="#">[가구] : 헌집 새집 베스트 가구</a><br>  
          <a href="#">[후기] : 배송이 잘못 됐어요 ㅠㅠ</a><br>  
        </div>  
      </div>  
</html>
```

```
</div>
</body>
</html>"""
```

```
In [3]: soup = BeautifulSoup(html_str, 'html.parser')
```

- select(태그명[속성])

```
In [4]: soup.find_all('a')
```

```
Out[4]: [<a href="#">패션잡화</a>,
<a href="#">주방용품</a>,
<a href="#">생활건강</a>,
<a href="#">DIY가구</a>,
<a href="#">[배송] : 무표배송 변경 안내 18.10.20</a>,
<a href="#">[전시] : DIY 가구 전시 안내 18.10.31</a>,
<a href="#">[판매] : 11월 특가 상품 안내 18.11.05</a>,
<a href="#">[레시피] : 살 안찌는 야식 만들기</a>,
<a href="#">[가구] : 헌집 새집 베스트 가구</a>,
<a href="#">[후기] : 배송이 잘못 됐어요 ㅋㅋ</a>]
```

```
In [5]: soup.select('a') # find_all(태그명)와 같은 결과 (리스트로 반환)
```

```
Out[5]: [<a href="#">패션잡화</a>,
<a href="#">주방용품</a>,
<a href="#">생활건강</a>,
<a href="#">DIY가구</a>,
<a href="#">[배송] : 무표배송 변경 안내 18.10.20</a>,
<a href="#">[전시] : DIY 가구 전시 안내 18.10.31</a>,
<a href="#">[판매] : 11월 특가 상품 안내 18.11.05</a>,
<a href="#">[레시피] : 살 안찌는 야식 만들기</a>,
<a href="#">[가구] : 헌집 새집 베스트 가구</a>,
<a href="#">[후기] : 배송이 잘못 됐어요 ㅋㅋ</a>]
```

```
In [6]: soup.select('a[href]')
```

```
Out[6]: [<a href="#">패션잡화</a>,
<a href="#">주방용품</a>,
<a href="#">생활건강</a>,
<a href="#">DIY가구</a>,
<a href="#">[배송] : 무표배송 변경 안내 18.10.20</a>,
<a href="#">[전시] : DIY 가구 전시 안내 18.10.31</a>,
<a href="#">[판매] : 11월 특가 상품 안내 18.11.05</a>,
<a href="#">[레시피] : 살 안찌는 야식 만들기</a>,
<a href="#">[가구] : 헌집 새집 베스트 가구</a>,
<a href="#">[후기] : 배송이 잘못 됐어요 ㅋㅋ</a>]
```

```
In [7]: [a.text for a in soup.select('a[href'])]
```

```
Out[7]: ['패션잡화',
'주방용품',
'생활건강',
'DIY가구',
'[배송] : 무표배송 변경 안내 18.10.20',
'[전시] : DIY 가구 전시 안내 18.10.31',
'[판매] : 11월 특가 상품 안내 18.11.05',
'[레시피] : 살 안찌는 야식 만들기',
'[가구] : 헌집 새집 베스트 가구',
'[후기] : 배송이 잘못 됐어요 ㅋㅋ']
```

- select(아이디선택자)

```
In [8]: soup.select('#mainMenuBox')
```

```
Out[8]: [<div id="mainMenuBox">
  <ul>
    <li><a href="#">패션잡화</a></li>
    <li><a href="#">주방용품</a></li>
    <li><a href="#">생활건강</a></li>
    <li><a href="#">DIY가구</a></li>
  </ul>
</div>]
```

- select(클래스선택자)

```
In [9]: soup.select('.box')
```

```
Out[9]: [<div class="box">
  <h4>공지사항</h4>
  <hr/>
  <a href="#">[배송] : 무표배송 변경 안내 18.10.20</a><br/>
  <a href="#">[전시] : DIY 가구 전시 안내 18.10.31</a><br/>
  <a href="#">[판매] : 11월 특가 상품 안내 18.11.05</a>
</div>,
<div class="box">
  <h4>커뮤니티</h4>
  <hr/>
  <a href="#">[레시피] : 살 안찌는 야식 만들기</a><br/>
  <a href="#">[가구] : 헌집 새집 베스트 가구</a><br/>
  <a href="#">[후기] : 배송이 잘못 됐어요 ㅋㅋ</a><br/>
</div>]
```

```
In [10]: soup.select('.box')[0]
```

```
Out[10]: <div class="box">
  <h4>공지사항</h4>
  <hr/>
  <a href="#">[배송] : 무표배송 변경 안내 18.10.20</a><br/>
  <a href="#">[전시] : DIY 가구 전시 안내 18.10.31</a><br/>
  <a href="#">[판매] : 11월 특가 상품 안내 18.11.05</a>
</div>
```

```
In [11]: soup.select('.box')[0].select('a')
```

```
Out[11]: [<a href="#">[배송] : 무표배송 변경 안내 18.10.20</a>,
  <a href="#">[전시] : DIY 가구 전시 안내 18.10.31</a>,
  <a href="#">[판매] : 11월 특가 상품 안내 18.11.05</a>]
```

```
In [12]: soup.select('.box')[0].select('a')[-1]
```

```
Out[12]: <a href="#">[판매] : 11월 특가 상품 안내 18.11.05</a>
```

```
In [13]: soup.select('.box')[0].select('a')[-1].text
```

```
Out[13]: '[판매] : 11월 특가 상품 안내 18.11.05'
```

- select(하위요소)

```
In [14]: soup.select('#mainMenuBox li')
```

```
Out[14]: [<li><a href="#">패션잡화</a></li>,
  <li><a href="#">주방용품</a></li>,
  <li><a href="#">생활건강</a></li>,
  <li><a href="#">DIY가구</a></li>]
```

```
In [15]: soup.select('.box a')
```



```
Out[15]: [<a href="#">[배송] : 무표배송 변경 안내 18.10.20</a>,
<a href="#">[전시] : DIY 가구 전시 안내 18.10.31</a>,
<a href="#">[판매] : 11월 특가 상품 안내 18.11.05</a>,
<a href="#">[레시피] : 살 안찌는 야식 만들기</a>,
<a href="#">[가구] : 헌집 새집 베스트 가구</a>,
<a href="#">[후기] : 배송이 잘못 됐어요 ㅋㅋ</a>]
```

```
In [16]: soup.select('div a')
```

```
Out[16]: [<a href="#">패션잡화</a>,
<a href="#">주방용품</a>,
<a href="#">생활건강</a>,
<a href="#">DIY가구</a>,
<a href="#">[배송] : 무표배송 변경 안내 18.10.20</a>,
<a href="#">[전시] : DIY 가구 전시 안내 18.10.31</a>,
<a href="#">[판매] : 11월 특가 상품 안내 18.11.05</a>,
<a href="#">[레시피] : 살 안찌는 야식 만들기</a>,
<a href="#">[가구] : 헌집 새집 베스트 가구</a>,
<a href="#">[후기] : 배송이 잘못 됐어요 ㅋㅋ</a>]
```

- select(직계자식)

```
In [17]: soup.select('div>a')
```

```
Out[17]: [<a href="#">[배송] : 무표배송 변경 안내 18.10.20</a>,
<a href="#">[전시] : DIY 가구 전시 안내 18.10.31</a>,
<a href="#">[판매] : 11월 특가 상품 안내 18.11.05</a>,
<a href="#">[레시피] : 살 안찌는 야식 만들기</a>,
<a href="#">[가구] : 헌집 새집 베스트 가구</a>,
<a href="#">[후기] : 배송이 잘못 됐어요 ㅋㅋ</a>]
```

```
In [20]: soup.select('#out_box > div')
```

```
Out[20]: [<div class="box">
<h4>공지사항</h4>
<hr/>
<a href="#">[배송] : 무표배송 변경 안내 18.10.20</a><br/>
<a href="#">[전시] : DIY 가구 전시 안내 18.10.31</a><br/>
<a href="#">[판매] : 11월 특가 상품 안내 18.11.05</a>
</div>,
<div class="box">
<h4>커뮤니티</h4>
<hr/>
<a href="#">[레시피] : 살 안찌는 야식 만들기</a><br/>
<a href="#">[가구] : 헌집 새집 베스트 가구</a><br/>
<a href="#">[후기] : 배송이 잘못 됐어요 ㅋㅋ</a><br/>
</div>]
```

```
In [21]: soup.select('#out_box > a')
```

```
Out[21]: []
```

- select(n번째요소)

```
In [23]: soup.select('div:nth-child(2)')
```

```
Out[23]: [<div>
  <table>
  <tr>
  <td></td>
  <td></td>
  <td></td></tr>
  <tr id="prdName">
  <td>솔로리스트<br/>걸리쉬 리본단화</td>
  <td>맥컬린<br/>그레이가보시스트랩 펌프스</td>
  <td>맥컬린<br/>섹슈얼인사이드펌프스</td></tr>
  <tr id="price">
  <td>100,000원</td><td>200,000원</td><td>120,000원</td></tr>
  </table>
</div>,
<div class="box">
<h4>커뮤니티</h4>
<hr/>
<a href="#">[레시피] : 살 안찌는 야식 만들기</a><br/>
<a href="#">[가구] : 현집 새집 베스트 가구</a><br/>
<a href="#">[후기] : 배송이 잘못 됐어요 ㅋㅋ</a><br/>
</div>]
```

```
In [24]: soup.select('div:nth-of-type(2)')
```

```
Out[24]: [<div>
  <table>
  <tr>
  <td></td>
  <td></td>
  <td></td></tr>
  <tr id="prdName">
  <td>솔로리스트<br/>걸리쉬 리본단화</td>
  <td>맥컬린<br/>그레이가보시스트랩 펌프스</td>
  <td>맥컬린<br/>섹슈얼인사이드펌프스</td></tr>
  <tr id="price">
  <td>100,000원</td><td>200,000원</td><td>120,000원</td></tr>
  </table>
</div>,
<div class="box">
<h4>커뮤니티</h4>
<hr/>
<a href="#">[레시피] : 살 안찌는 야식 만들기</a><br/>
<a href="#">[가구] : 현집 새집 베스트 가구</a><br/>
<a href="#">[후기] : 배송이 잘못 됐어요 ㅋㅋ</a><br/>
</div>]
```

```
In [26]: soup.select('#out_box > div')
```

```
Out[26]: [<div class="box">
  <h4>공지사항</h4>
  <hr/>
  <a href="#">[배송] : 무표배송 변경 안내 18.10.20</a><br/>
  <a href="#">[전시] : DIY 가구 전시 안내 18.10.31</a><br/>
  <a href="#">[판매] : 11월 특가 상품 안내 18.11.05</a>
  </div>,
  <div class="box">
  <h4>커뮤니티</h4>
  <hr/>
  <a href="#">[레시피] : 살 안찌는 야식 만들기</a><br/>
  <a href="#">[가구] : 현집 새집 베스트 가구</a><br/>
  <a href="#">[후기] : 배송이 잘못 됐어요 ㅋㅋ</a><br/>
  </div>]
```

```
In [25]: soup.select('#out_box > div:nth-of-type(2)')
```

```
Out[25]: [<div class="box">
  <h4>커뮤니티</h4>
  <hr/>
  <a href="#">[레시피] : 살 안찌는 야식 만들기</a><br/>
  <a href="#">[가구] : 헌집 새집 베스트 가구</a><br/>
  <a href="#">[후기] : 배송이 잘못 됐어요 ππ</a><br/>
</div>]
```

```
In [27]: soup.select('a')
```

```
Out[27]: [<a href="#">패션잡화</a>,
  <a href="#">주방용품</a>,
  <a href="#">생활건강</a>,
  <a href="#">DIY가구</a>,
  <a href="#">[배송] : 무표배송 변경 안내 18.10.20</a>,
  <a href="#">[전시] : DIY 가구 전시 안내 18.10.31</a>,
  <a href="#">[판매] : 11월 특가 상품 안내 18.11.05</a>,
  <a href="#">[레시피] : 살 안찌는 야식 만들기</a>,
  <a href="#">[가구] : 헌집 새집 베스트 가구</a>,
  <a href="#">[후기] : 배송이 잘못 됐어요 ππ</a>]
```

```
In [29]: soup.select('a:nth-child(3)')
```

```
Out[29]: [<a href="#">[배송] : 무표배송 변경 안내 18.10.20</a>,
  <a href="#">[레시피] : 살 안찌는 야식 만들기</a>]
```

```
In [30]: soup.select('a:nth-of-type(3)')
```

```
Out[30]: [<a href="#">[판매] : 11월 특가 상품 안내 18.11.05</a>,
  <a href="#">[후기] : 배송이 잘못 됐어요 ππ</a>]
```

- select(속성선택)

```
In [57]: html_str = '''
  <a href='https://www.google.com' target="_blank"> Google </a>
  <a href='https://www.daum.net'> Daum </a>
  <a href='https://www.naver.com'> Naver </a>
  <a href='http://tstory.com'>Tstory</a>
  <a href='report.pdf'>보고서</a>
  <a href='data.csv'>데이터</a>
  <input type='text'>
  <input type='password'>
  <input type='text'>
  '''
```

```
In [58]: soup = BeautifulSoup(html_str, 'html.parser')
```

```
In [37]: links = soup.select('a[href]')
  links
```

```
Out[37]: [<a href="https://www.google.com" target="_blank">Google</a>,
  <a href="https://www.daum.net">Daum</a>,
  <a href="https://www.naver.com">Naver</a>,
  <a href="http://tstory.com">Tstory</a>,
  <a href="report.pdf">보고서</a>,
  <a href="data.csv">데이터</a>]
```

```
In [39]: for link in links:
  print(link["href"])
```

```
https://www.google.com
https://www.daum.net
https://www.naver.com
http://tstory.com
report.pdf
data.csv
```

```
In [40]: # 안전한 방식 get() 사용
for link in links:
    print(link.get("href"))
```

```
https://www.google.com
https://www.daum.net
https://www.naver.com
http://tstory.com
report.pdf
data.csv
```

- select(속성값 선택)

```
In [42]: soup.select('input[type="text"]')
```

```
Out[42]: [<input type="text"/>, <input type="text"/>]
```

```
In [43]: soup.select('a[target="_blank"]')
```

```
Out[43]: [<a href="https://www.google.com" target="_blank">Google</a>]
```

```
In [45]: soup.select('a[target="_blank"]')[0].get("href")
```

```
Out[45]: 'https://www.google.com'
```

- select(속성값 부분검색) : 특정문자열로 시작

```
In [46]: soup.select('a[href]')
```

```
Out[46]: [<a href="https://www.google.com" target="_blank">Google</a>,
<a href="https://www.daum.net">Daum</a>,
<a href="https://www.naver.com">Naver</a>,
<a href="http://tstory.com">Tstory</a>,
<a href="report.pdf">보고서</a>,
<a href="data.csv">데이터</a>]
```

```
In [49]: # href속성 값이 https로 시작하는 태그 찾기 : ^ 기호
soup.select('a[href^="https"]')
```

```
Out[49]: [<a href="https://www.google.com" target="_blank">Google</a>,
<a href="https://www.daum.net">Daum</a>,
<a href="https://www.naver.com">Naver</a>]
```

```
In [50]: # href속성 값이 .pdf로 끝나는 태그 찾기 : $ 기호
soup.select('a[href$=".pdf"]')
```

```
Out[50]: [<a href="report.pdf">보고서</a>]
```

```
In [53]: # href속성 값이 com을 포함하는 태그 찾기 : * 기호
soup.select('a[href*="com"]')
```

```
Out[53]: [<a href="https://www.google.com" target="_blank">Google</a>,
<a href="https://www.naver.com">Naver</a>,
<a href="http://tstory.com">Tstory</a>]
```

텍스트 / 속성값 추출

- 태그 자체보다 텍스트 추출을 가져오는 경우가 많음
- `태그.get_text()` 를 이용한 텍스트 추출
 - <https://www.crummy.com/software/BeautifulSoup/bs4/doc/#get-text>
- `태그.get(속성명)` 을 이용한 속성값 추출

```
In [54]: soup.select_one('a[href*="com"]')
```

```
Out[54]: <a href="https://www.google.com" target="_blank">Google</a>
```

```
In [59]: soup.select_one('a[href*="com"]').get_text()
```

```
Out[59]: ' Google '
```

```
In [61]: soup.select_one('a[href*="com"]').text
```

```
Out[61]: ' Google '
```

```
In [60]: soup.select_one('a[href*="com"]').get_text(strip=True)
```

```
Out[60]: 'Google'
```

```
In [56]: soup.select_one('a[href*="com"]').get('href')
```

```
Out[56]: 'https://www.google.com'
```

문제.

다음 사이트(<https://www.daum.net/>) 지도, 금융, 메일, 스포츠, 날씨, 사전, 카페로 이동하는 링크 주소 추출하기

```
In [ ]:
```
